

dmap() manual

Activation of the function

If you have all the stuff on GitHub in a particular folder and dmap.R is also there, then it will be able to access the files it needs, which are specified in the beginning of the script. Activate the dmap() function and its required objects by writing `source("dmap.R")` after having set the working directory to the relevant folder.

Arguments

k: the number clusters desired, for instance `k = 4` (default `k = 5`).

tree: the clusterization method. Options are WardD, WardD2, complete, UPGMA, WPGMA (default being UPGMA). Note that quotation marks are not used, because the trees are objects.

lonlim, latlim: specify a subarea setting these (default is the entire area covered by DARJa). For instance, `lonlim=c(27.8,31.5)`, `latlim=c(58,60)` shows the diversity in the NW and `lonlim=c(40,45)`, `latlim=c(58,60)` shows the Chuxloma ‘island’.

CITIES: if set to TRUE, shows locations and names of selected anchoring cities for any plot using the default GEOPLOT layout. The data for the cities are in cities.RData, which contains the object called cities. Modifying this object would allow for adding or removing cities to be plotted.

AREA: choose one among 9 cells in a 3 x 3 grid to zoom in on; options: "SW", "CW", "SW", "NC", "CC", "SC", "NE", "CE", "SE".

lofvals: if set to TRUE, lof scores will be employed as a way to identify outliers. lof scores are scores of how much a location diverges geographically from its fellows cluster members (=local outlier factor). minpnts is the number of nearest neighbors used in defining the local neighborhood of a point (includes the point itself). It needs to be above 63 to recognize the Chuxloma outliers. The lofcutoff could be lowered to include more of the points in the north, but then it will also include some dots that just happen to be a bit removed from fellow cluster members for geographical reasons.

SVM: if set to TRUE, a support vector machine will be employed as a way to identify outliers; they are identified as such if they are not predicted by the model. It doesn’t work so well, perhaps it might work better with some parameter tuning.

DBSCAN: if set to TRUE; this identifies noisy points, which can be considered outliers; results are rather similar to lof.

KNN: if set to TRUE, this counts the number of neighbors belonging to the same group; if this is below the SGC (same group count) threshold the location is counted as an outlier; it seems non-trivial to tune parameters to get a meaningful result

GMM: if set to TRUE, a Gaussian Mixture Models will produce a log-likelihood for the classification of a given point in its particular cluster; if this log-likelihood falls beneath a certain max probability (MP) cutoff, it is defined as an outlier; somehow doesn't work well.

LINES: if set to TRUE, the area is separated into a 3 x 3 grid by lines; only works when GEOPLOT=FALSE.

GEOPLOT: if set to TRUE (the default), displays a ggplot2 map; if set to FALSE, displays a map without geographical projection, lon and lat being treated as plain Cartesian coordinates. In the GEOPLOT=FALSE mode, labels showing IDs are added automatically when looking for outliers and zooming in on a certain area.

FILL: color of land.

SCALE: "L" for high resolution (showing more rivers); "M" for medium resolution (fewer rivers); "S" for low resolution (nearly no rivers).

GENERICPLOT: if set to TRUE, the function just plots the locations of DARJa. Anchoring cities are always included in this plot. Tip: when exporting the image a good width to choose is 800.

MARGIN: adjusts the margin for the purpose of GENERICPLOT.

POINTSIZ: adjusts the point size for the purpose of GENERICPLOT.

CITYSIZE: adjusts the size of symbol for anchoring cities for the purpose of GENERICPLOT.

CITYSHAPE: adjusts the symbol for anchoring cities for the purpose of GENERICPLOT.

BIOMEPL: if set to TRUE, display the biomes of the DARJa area, also outputting a legend to the console.

FT: stands for 'feature'. If this is some entire feature (e.g., "M_6"), a map will be created showing the distribution of values of that feature. Normally multiple values could not be shown—when they occur, some random value would overwrite the others. Therefore there is a JITTER parameter set by default to 0.05 (degrees), which *only* applies when FT is a whole feature (not if it is a value); if FT is some feature value (e.g., "M_6_1"), the distribution of this value will be shown

JITTER: randomly jitters points by the number of degrees indicated, the default being 0.05; this only happens when FT is set to some feature. When set to 0, jittering is turned off.

ALPHA: a transparency parameter for points; the closer to zero, the more transparent. This only applies when FT is some feature. The default is set to 1 (non-transparent), i.e., it will not apply unless explicitly set.

Detail on outliers

When one of the outlier-identifying functions is turned on, the following information will be added to the data frame with outgroup information: outgroup, id_inmate, orig_id_inmate,

id_outmate, orig_id_outmate, hamming_inmate, hamming_outmate. A text file called outliers.txt will contain all information.

Detail on anchoring cities in the generic plot

A data frame called cities used for plotting some major cities in the area was prepared in the following way, which could be modified to make change to the cities chosen for display:

```
library(toponym) - see https://github.com/Lennart05/toponym  
  
top(strings=c("Saint Petersburg", "Vologda", "Pskov", "Moscow",  
"Nizhniy Novgorod", "Smolensk", "Tambov", "Belgorod", "Chukhloma",  
"Velikiy Novgorod", "Velikiy Ustyug"), countries="RU",  
name="cities_all")
```

that produces a data frame cities_all from geonames.org, but with some cities whose names are synonymous, so the right ones are filtered by their geonames IDs

```
ids <- c(498817, 472459, 504341, 524901, 520555, 491687, 484646,  
578072, 567695, 519336, 476062)  
  
w_ids <- match(ids, cities_all$geonameid)  
  
cities <- cities_all[w_ids,c("name", "latitude", "longitude",  
"group")]
```

There is a column called group, which is reused here with the same name but just given a factor of 1 as values

```
cities$group <- as.factor(1)  
  
save(cities, file="cities.RData")
```